

Full Stack Developer Round 1 Assignment

Full Stack + Applied AI | IIFL Finance

Candidate timebox: 2–3 hours maximum

AI-assisted development encouraged

This is intentionally a small prototype exercise. We are evaluating how effectively you use modern development tools, make engineering decisions, and validate AI-generated code—not how much functionality you can build after the timebox.

1. Purpose

This first-round exercise is designed to assess practical full stack engineering ability across a frontend, backend API, persistence, and a small AI integration. We also want to see basic judgment around reliability, privacy, and production readiness.

Use of ChatGPT, Claude, Cursor, Copilot, Gemini, or similar AI coding tools is explicitly permitted and encouraged. Briefly note which tools you used and how you reviewed or modified their output.

2. Assignment

Build a Small AI Content Assistant

Build a simple application where a user submits a block of text—for example, a note, article draft, or meeting transcript. The application should use an LLM to generate:

- a concise summary; and
- exactly three relevant tags.

The result should be stored and displayed in the application.

Expected user flow

1. User enters text in the frontend and submits it.
2. Backend validates the request and calls an LLM.
3. Backend parses the AI response into a predictable structure.
4. Original text, summary, and tags are stored in a database.
5. Frontend displays the saved entries and allows the user to open one entry.

Expected AI output

Return or internally normalize the model response into a structure similar to:

```
{
  "summary": "A concise summary of the submitted text.",
  "tags": ["tag-1", "tag-2", "tag-3"]
}
```

3. Minimum Requirements

Keep the implementation deliberately small. The following is sufficient.

Area	Minimum expectation
Frontend	Any modern frontend framework. One input form, loading/error state, and a list/detail view of saved entries.
Backend	Python using any framework of your choice. Expose the small REST API needed by your UI.
Persistence	SQLite is sufficient. Persistence must work across application restarts; it should not be mocked.
AI integration	Call one real LLM API. Shape the prompt and parse the response into summary + three tags.
Failure handling	Gracefully handle at least one AI failure case such as timeout, invalid JSON, or malformed output.
Testing	Add at least one useful automated test for non-AI logic or an API endpoint.

Suggested API shape

You may design the API differently. A minimal implementation could use only:

- `POST /entries` — submit text, generate AI output, persist the entry.
- `GET /entries` — list saved entries.
- `GET /entries/{id}` — optional; useful if your UI has a detail view.

Basic validation expected

A strong submission does not need elaborate safeguards. Demonstrating one or two of the following is enough:

- reject empty input;
- set a reasonable maximum input length;
- validate that the AI returned a summary and exactly three tags;
- surface a useful error instead of crashing when the model call fails.

4. Explicitly Out of Scope

To protect the 2–3 hour timebox, **do not spend time** on the following unless you already have them for free through your tooling:

- polished visual design or complex responsive layouts;
- authentication or role-based access control;
- cloud deployment, CI/CD, Docker, Kubernetes, or infrastructure-as-code;
- vector databases, RAG pipelines, agents, or multi-step orchestration;
- extensive test suites;
- analytics, observability dashboards, or production monitoring;
- a video walkthrough;
- a separate system-design document or long written evaluation.

If you would add any of these in production, mention it briefly in the README rather than implementing it.

5. Deliverables

Submit one public repository (preferably GitHub) containing:

- frontend and backend source code;
- database/schema setup required to run locally;
- `.env.example` with the required AI configuration names only—never commit credentials;
- a short `README.md` containing setup instructions and one screenshot showing the working end-to-end flow.

README - keep it short

Your README should also include concise answers to the following questions:

1. **Architecture:** How does the request flow from frontend to backend, LLM, and database? *Maximum 5–8 lines.*
2. **AI choice:** Which model/provider did you use, and why? *Maximum 3–5 lines.*
3. **Reliability:** What happens if the model is slow or returns malformed output? *Maximum 3–5 lines.*
4. **Privacy:** What data is sent to the LLM provider, and what would you avoid sending in a financial-services environment? *Maximum 3–5 lines.*
5. **Production next step:** If given one more day, what are the top three improvements you would make? *Maximum 3 bullets.*
6. **AI coding tools:** Which AI tools did you use while building the assignment, and how did you validate their output? *Maximum 3–5 lines.*

6. Suggested Time Budget

This is guidance, not a scoring criterion.

Activity	Focus	Suggested time
Setup & architecture	Scaffold app, choose schema/API/model	15–20 min
Core implementation	Frontend, backend, persistence, LLM call	75–100 min
Validation & test	Error path, one automated test, quick cleanup	20–30 min
README & screenshot	Setup instructions and short judgment questions	20–30 min
Total		2–3 hours

7. Evaluation Criteria

Criterion	Weight	What we are looking for
End-to-end execution	35%	A working frontend, backend, persistence layer, and AI flow.
AI integration quality	20%	Sensible prompt/output handling and graceful handling of AI failure.
Backend & data design	15%	Clear API design, understandable code, and appropriate persistence.
Frontend quality	10%	Clean integration, basic UX states, and understandable component structure.
Engineering judgment	10%	Practical trade-offs around reliability, privacy, and next steps.
Code quality & test	10%	Legible structure plus at least one meaningful test.

8. What We Do Not Reward

Extra complexity is not automatically better. Candidates will not receive additional credit merely for using more frameworks, building a polished UI, deploying to the cloud, adding a vector database, or creating a large test suite. We value a small solution that works, is easy to understand, and demonstrates sound judgment.

9. Next Step

Shortlisted candidates will move to Round 2, where you will walk through your submission, make a small change or extension live, and discuss trade-offs such as scaling, cost, latency, privacy, and production architecture.

We are looking forward to seeing how you think, build, and use modern AI-assisted development tools effectively.